

	PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINHAS GERAIS Instituto de Ciências Exatas e Informática (ICEI) Departamento de Sistemas de Informação e Engenharia de Software Graduação em Engenharia de Software RELATÓRIO TÉCNICO – MÉTODO ATAM <i>(Architecture Tradeoff Analysis Method)</i>	
	PROFESSOR: CLEITON SILVA TAVARES · CRISTIANO DE MACÊDO NETO · JOÃO PAULO CARNEIRO ARAMUNI	
	DISCIPLINA: TRABALHO INTERDISCIPLINAR: APLICAÇÕES DISTRIBUÍDAS PERÍODO: 5º	
	PROJETO: Biblioio	DATA: 19/06/2026

1. Resumo Executivo

O Biblioio é uma plataforma de comunidade literária web e mobile voltada a leitores ativos brasileiros, que transforma o histórico de leitura em direcionamento inteligente e conexões reais entre leitores. O sistema integra seis trilhas algorítmicas independentes de recomendação personalizada (grafos Neo4j, Thompson Sampling, decay exponencial, repetição espaçada), feed social com posts e reviews, comunidades com chat em tempo real via WebSocket/STOMP, DNA Literário e o assistente conversacional Bibo (Google Gemini). O backend é implementado em Spring Boot 4 (Java 25) com arquitetura Hexagonal em monólito modular de 11 domínios; o frontend em Next.js 16 / React 19; e o app mobile em Flutter 3.11 com padrão offline-first.

A avaliação ATAM priorizou cinco atributos de qualidade diretamente ligados aos objetivos de negócio: Desempenho/Escalabilidade, Segurança, Modificabilidade, Disponibilidade/Resiliência e Usabilidade. Os principais achados revelaram que a arquitetura suporta carga com excelência — 24/24 testes K6 aprovados, 0% de falhas sistêmicas — e que as decisões de design (cache Redis, bancos especializados, isolamento por eventos assíncronos e offline-first) atendem consistentemente aos direcionadores de negócio.

Os riscos mais críticos identificados são: (R-01) contenção de concorrência em operações de Votação e JoinRequest sob alta carga; (R-02) ponto único de falha em SecurityConfig para autorização de endpoints; (R-03) VM GCE própria do OpenSearch em produção como componente não gerenciado; (R-04) ausência de resolução de conflitos

em sincronização offline multi-dispositivo no app mobile. Recomenda-se como ação prioritária a adoção de lock otimista (@Version) nas entidades de alta contenção e a implementação de testes de contrato obrigatórios para novos endpoints públicos declarados em SecurityConfig.

2. Objetivos de Negócio e Direcionadores Arquiteturais

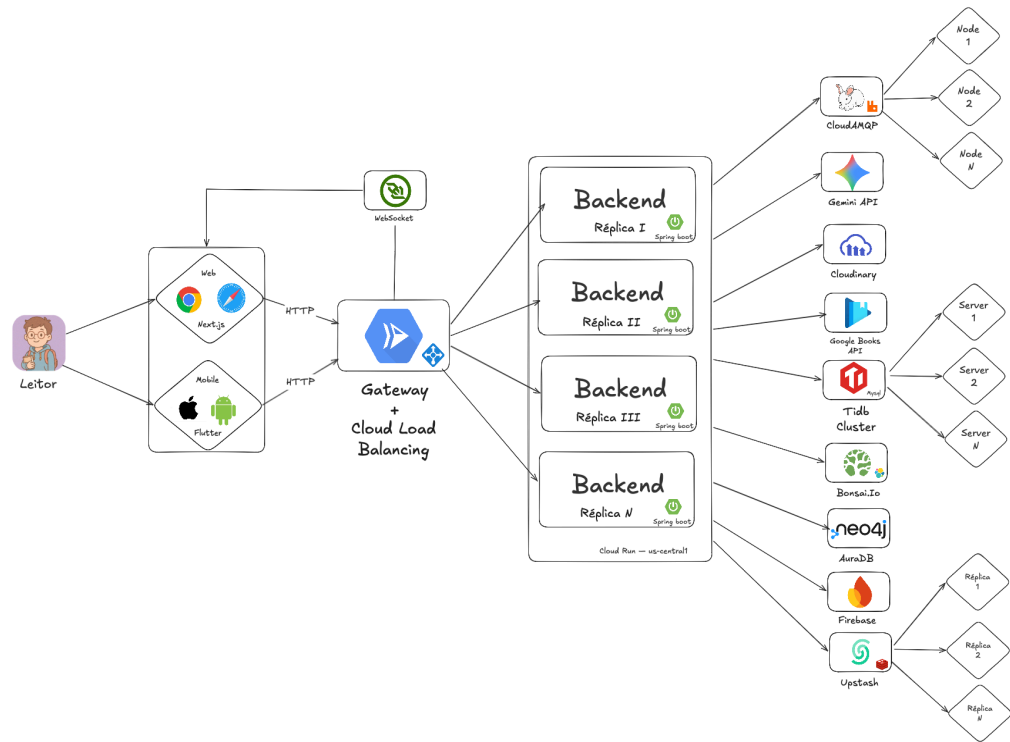
O Biblío foi criado para reverter o abandono da leitura no Brasil — 53% dos brasileiros não leram sequer parte de um livro nos últimos 3 meses (Retratos da Leitura, 2024) — oferecendo uma experiência integrada de descoberta literária, interação social e identidade do leitor. A arquitetura deve garantir engajamento contínuo, confiabilidade das interações sociais em tempo real, segurança dos dados dos usuários e capacidade de evolução sustentada do produto.

Principais direcionadores de qualidade identificados:

- Direcionador 1: Desempenho e Escalabilidade — o sistema deve responder com baixa latência e sem falhas sistêmicas sob carga concorrente realista, garantindo resposta fluida nas 6 trilhas de recomendação, no feed social e no chat em tempo real.
- Direcionador 2: Segurança e Proteção de Dados — autenticação JWT + OAuth Google, sanitização contra XSS em todo conteúdo gerado por usuário, rate limiting do assistente Bibó e gestão de 40+ secrets via Google Secret Manager são pré-requisitos de confiança para uma rede social.
- Direcionador 3: Modificabilidade e Evolução Contínua — os 11 módulos de domínio devem ser independentes entre si, permitindo adição de novas trilhas de recomendação ou novos módulos sem reescrita dos existentes, sustentando o ritmo de sprints do curso.
- Direcionador 4: Disponibilidade e Resiliência Operacional — fallbacks explícitos para serviços externos (OpenSearch → MySQL FULLTEXT, Google Books API → catálogo local) e operação offline-first no app mobile garantem continuidade mesmo em degradação parcial de infraestrutura.

3. Topologia e Abordagem da Arquitetura

O Biblio segue a arquitetura Hexagonal (Ports & Adapters) em monólito modular, composto por três produtos integrados: backend Spring Boot, frontend Next.js e app mobile Flutter. Os módulos de domínio nunca se importam diretamente — toda comunicação inter-módulo ocorre via eventos assíncronos (RabbitMQ, padrão Outbox) ou interfaces. Serviços gerenciados externos (TiDB Cloud, Upstash Redis, CloudAMQP, Neo4j Aura, Bonsai.io/GCE) são acessados exclusivamente pelo backend via adapters.



Componente / Camada	Tecnologia Proposta	Papel Estrutural e Mecanismo de Comunicação
Camada de Apresentação Web	Next.js 16 · React 19 · TypeScript · Tailwind CSS · Radix UI · Framer Motion	SPA com App Router que realiza chamadas HTTP REST autenticadas com JWT Bearer e abre conexões WebSocket/STOMP para o chat das comunidades e SSE para o assistente Bibó. Deploy automático na Vercel a cada push em main.
App Mobile	Flutter 3.11 · Dart 3.11 · BLoC · Drift (SQLite) · Hive · Dio · GoRouter · FlutterSecureStorage	App offline-first para Android/iOS. Repository local-first (Drift/Hive); AuthInterceptor (Dio) injeta JWT e renova em 401; RetryInterceptor com backoff exponencial; 13 features independentes com padrão BLoC/Repository/Datasource.

Backend Central (API REST + WebSocket)	Spring Boot 4 · Java 25 · Arquitetura Hexagonal · 11 módulos	Motor de regras de negócio puras sem dependência de framework. Expõe REST controllers e WebSocket handlers como portas de entrada. Comunica-se com adapters de persistência, cache, mensageria e APIs externas. Nunca há importação direta entre módulos de domínio.
Cache Distribuído	Redis 7.4 (Upstash em produção)	Buffer em memória para dados de alta leitura: trending (TTL 15 min), feed sliding window (200 itens por usuário), parâmetros bayesianos Thompson Sampling (TTL 90 dias), histórico de conversas do Bibi (TTL configurável). Política cache-null-values=false para prevenir cache poisoning.
Persistência Relacional	MySQL 8.4 (TiDB Cloud Serverless) + HikariCP	Armazenamento canônico e transacional de todas as entidades do domínio. Outbox Pattern: eventos publicados dentro de @Transactional, garantindo consistência entre entidade e mensagem. schema.sql roda a cada startup (spring.sql.init.mode=always).
Banco de Grafos	Neo4j 5.18 (Aura Free) via neo4j-java-driver com Cypher raw	Armazena relacionamentos (:User)-[:READ]->(:Book) para as trilhas T1 (BecauseYouRead) e T5 (SimilarAuthors). Navegação de até 2 níveis de relacionamento. Fallback automático para MySQL FULLTEXT quando indisponível.
Busca Full-Text	OpenSearch 2.18 (Bonsai.io portfolio / GCE VM e2-small produção)	Índice derivado para busca de livros (título, autor, ISBN) e usuários (username). OpenSearchIndexCleanupService realiza limpeza/reconciliação semanal com MySQL. Fallback para MySQL FULLTEXT quando indisponível.
Mensageria / Broker WebSocket	RabbitMQ 4.0 (CloudAMQP Little Lemur) + STOMP	Desacopla módulos via Outbox Pattern (eventos assíncronos, idempotência por event_id). Também serve como broker WebSocket/STOMP para o chat das comunidades — FanoutExchange distribui mensagens entre todas as instâncias Cloud Run ativas via WebSocketMessageBroadcastAdapter.
Notificações / APIs Externas	Firebase FCM · Cloudinary · Google Gemini · SendGrid · Google Books API	Firebase FCM: push notifications para mobile. Cloudinary: upload assíncrono de imagens com CDN. Google Gemini (via Spring AI): LLM do assistente Bibi com histórico no Redis e rate limit de 20 req/min via Bucket4j. SendGrid: e-mails transacionais de redefinição de senha.

Infraestrutura e Deploy	Google Cloud Run · Cloud Build · GitHub Actions · Secret Manager · Artifact Registry	Dois ambientes independentes: portfolio (0–2 instâncias, 1Gi/1vCPU, hiberna sem tráfego) e producao (1–10 instâncias, 2Gi/2vCPU, sempre ativa). Pipeline CI/CD: push em prod → GitHub Actions espelha → Cloud Build → 4 steps (build/push/deploy) em ~12 min sem downtime.
Observabilidade	Prometheus · Grafana · Micrometer · K6	Métricas expostas via /actuator/prometheus: histograma de latência HTTP (p50/p95/p99), pool HikariCP, consumer lag RabbitMQ, contadores dos algoritmos de recomendação. Dashboard Grafana pré-configurado (config/grafana.json). 24 suítes K6 com thresholds de SLA por endpoint.

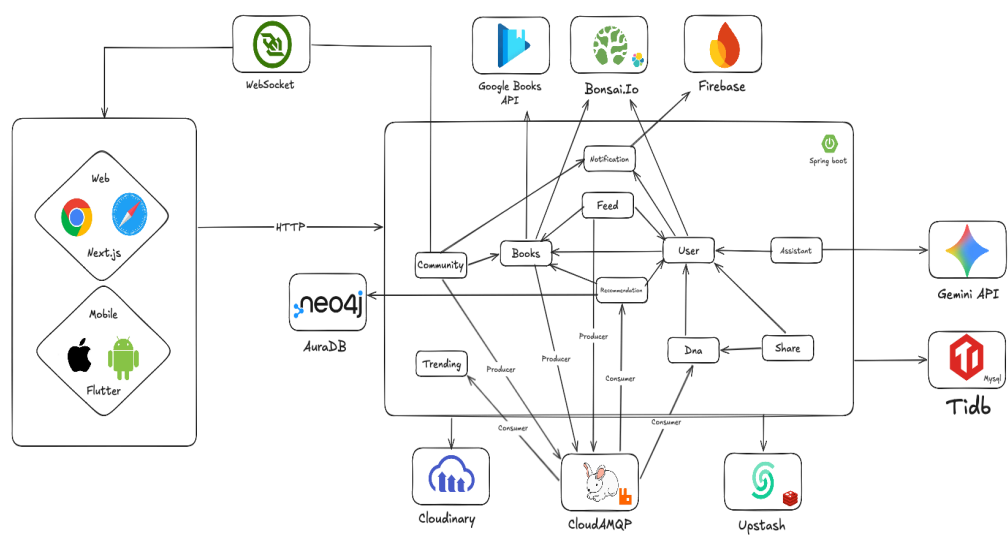


Figura 1 — Arquitetura de Domínio do Biblioo. Fonte: os próprios autores.

4. Árvore de Utilidade e Inventário de Cenários

A árvore de utilidade abaixo prioriza os cenários por Importância para o negócio e Dificuldade arquitetural. Os cenários com prioridade (Alta/Alta) e (Alta/Média) foram selecionados para análise detalhada na Seção 5. Cada cenário contém: estímulo (o que dispara), ambiente (contexto) e resposta esperada mensurável.

ID	Atributo de Qualidade	Descrição do Cenário (Estímulo, Ambiente e Resposta)	Prioridade (Negócio / Dificuldade)
C-01	Desempenho / Escalabilidade	Estímulo: Carga de 80 VUs simultâneos no endpoint /recommendations/* (6 trilhas em batch paralelo). Ambiente: Sistema em operação normal de produção. Resposta esperada: $p95 \leq 1.200ms$ (CatalogSurprise/SimilarAuthors), $p95 \leq 800ms$ nas demais trilhas; taxa de erro HTTP < 1% em todos os endpoints.	Alta / Alta
C-02	Disponibilidade / Resiliência	Estímulo: A instância OpenSearch (Bonsai.io no portfolio / GCE VM no producao) fica indisponível durante um pico de uso no endpoint /books/search. Ambiente: Degradação parcial de infraestrutura — serviço externo fora do controle da aplicação. Resposta esperada: /books/search continua retornando 200 OK com resultados via fallback FULLTEXT MySQL, sem propagar 5xx ao cliente.	Alta / Alta
C-03	Segurança	Estímulo: (a) Requisição HTTP sem Authorization: Bearer válido a endpoint protegido; (b) Leitor publica post/comentário/bio contendo <code><script>alert(1)</script></code> . Ambiente: Sistema em operação normal e sob tentativa de abuso. Resposta esperada: (a) 401 Unauthorized imediato; (b) markup removido antes da persistência — conteúdo seguro para renderização.	Alta / Alta
C-04	Modificabilidade	Estímulo: Desenvolvedor precisa adicionar a trilha T7 (recomendações sazonais) reagindo a eventos BOOK_FINISHED já publicados pelo módulo books/shelf. Ambiente: Tempo de desenvolvimento — sprint normal do projeto. Resposta esperada: T7 implementada como módulo isolado com seu próprio consumer RabbitMQ; nenhuma linha dos outros 10 módulos é alterada; parâmetros ajustáveis via @Value sem recompilação.	Alta / Média
C-05	Usabilidade / Disponibilidade Percebida	Estímulo: Leitor sem conexão à internet abre o app Flutter e atualiza a página atual de um livro em sua estante (PATCH /shelves/{id}/items/{itemId}/progress). Ambiente: App mobile sem conectividade de rede (transporte público, área sem cobertura). Resposta esperada: UI reflete a atualização imediatamente (via	Alta / Alta

		Drift/SQLite local); ao reconectar, RetryInterceptor sincroniza automaticamente sem ação manual do usuário.	
C-06	Disponibilidade / Chat em Tempo Real	Estímulo: Mensagem enviada por membro conectado na instância A do Cloud Run deve ser entregue a membro conectado na instância B, dado que o Cloud Run escala horizontalmente. Ambiente: Sistema em produção com múltiplas réplicas ativas (até 10 instâncias) e session-affinity habilitado. Resposta esperada: Mensagem entregue a todos os membros da comunidade via RabbitMQ FanoutExchange (WebSocketMessageBroadcastAdapter → CommunityBroadcastConsumer em cada instância), sem perda.	Alta / Média
C-07	Segurança / Proteção de Secrets	Estímulo: Deploy de nova revisão no Cloud Run sem que nenhum dos 40+ secrets de produção seja exposto em código-fonte, variáveis de ambiente em texto claro ou logs da aplicação. Ambiente: Pipeline CI/CD automatizado (Cloud Build → Cloud Run). Resposta esperada: Todos os secrets injetados exclusivamente via Google Secret Manager (--set-secrets) como variáveis de ambiente criptografadas — nenhum valor sensível aparece em código ou logs.	Alta / Baixa

5. Inventário Analítico Rastreável (Saídas do ATAM)

Os achados abaixo foram derivados do cruzamento dos 7 cenários da árvore de utilidade com as abordagens arquiteturais identificadas na etapa 4 (Hexagonal, Event-Driven, Cache-Aside, JWT stateless, offline-first, CI/CD automatizado). Cada item é rastreável ao(s) cenário(s) que o motivou.

Riscos Identificados (R)

- R-01:** Contenção de concorrência em Voting e JoinRequest sob alta carga

Cenários afetados: C-01 | Componente crítico: Módulo community (Spring Boot — entidades Voting, JoinRequest)

Descrição técnica: Sob alta contenção (múltiplos usuários votando ou solicitando entrada simultaneamente), operações concorrentes sobre o mesmo registro de Voting ou JoinRequest podem colidir. O módulo não implementa lock otimista (@Version), o que significa que a última escrita vence silenciosamente ou que uma das requisições recebe um erro de constraint inesperado. Nos testes K6 (stress,

500 VUs), o endpoint de votação apresentou 0,76% de falha sob contenção fabricada — aceitável em testes, mas potencialmente mais alto em produção real com grande número de comunidades ativas. O atributo de Desempenho e integridade funcional são comprometidos.

- **R-02:** Ponto único de falha na declaração de endpoints públicos em SecurityConfig
Cenários afetados: C-03 | Componente crítico: SecurityConfig.java (Spring Security)

Descrição técnica: A lista de endpoints públicos em SecurityConfig é o único ponto de decisão sobre o que fica exposto sem autenticação. Uma declaração incorreta — como marcar um endpoint sensível como `.permitAll()` por engano durante o desenvolvimento de um novo módulo — expõe aquele recurso sem nenhuma camada de defesa redundante, pois o `JwtAuthenticationFilter` não processa a requisição se o path já foi liberado. Não há testes automatizados de contrato que verifiquem se endpoints sensíveis retornam 401 ao chamar sem token — um regression test ausente que seria a mitigação mais direta.

- **R-03:** VM GCE própria do OpenSearch em produção como componente não gerenciado

Cenários afetados: C-02 | Componente crítico: VM biblio-infra (GCE e2-small, OpenSearch 2.18)

Descrição técnica: Em produção, o OpenSearch roda em uma VM GCE própria (biblio-infra, e2-small, `DISABLE_SECURITY_PLUGIN=true`, HTTP :9200 em rede VPC interna) — diferentemente do ambiente portfolio (Bonsai.io, gerenciado). Essa VM não tem auto-healing, não tem backup automatizado e depende de operação manual (SSH, docker restart) para recuperação. Uma falha de disco ou OOM na VM interrompe o OpenSearch em produção até intervenção manual, ativando o fallback FULLTEXT e degradando a experiência de busca por um tempo indeterminado. O custo de disponibilidade recai inteiramente sobre a equipe.

- **R-04:** Ausência de resolução de conflitos em sincronização offline multi-dispositivo
Cenários afetados: C-05 | Componente crítico: ShelfRepository / RetryInterceptor (Flutter mobile)

Descrição técnica: O padrão offline-first persiste alterações localmente (Drift/SQLite) e as sincroniza ao reconectar via RetryInterceptor. No entanto, não há lógica de merge/conflict-resolution: se o mesmo ShelfItem for alterado offline em dois dispositivos do mesmo usuário antes da sincronização, a última requisição a chegar ao servidor vence silenciosamente — a outra alteração é descartada sem notificação ao usuário. Embora seja um cenário de baixa frequência, a perda silenciosa de dados compromete a confiabilidade percebida do app.

Pontos de Não-Risco (NR)

- **NR-01:** Outbox Pattern + idempotência por event_id eliminam mensagens duplicadas e inconsistências inter-módulo

Cenários afetados: C-04, C-06

Descrição técnica: Toda publicação no RabbitMQ ocorre dentro de @Transactional via OutboxEvent, garantindo que nenhuma mensagem seja publicada sem o commit da entidade relacionada. Cada consumer verifica o event_id antes de processar, garantindo idempotência — eventos reentregues por falha de rede não causam efeitos colaterais duplicados. Esta é uma decisão sólida que neutraliza uma categoria inteira de bugs de consistência eventual em sistemas event-driven sem necessidade de sagas complexas.

- **NR-02:** Fallback automático OpenSearch → MySQL FULLTEXT elimina erros 5xx originados por busca

Cenários afetados: C-02

Descrição técnica: O módulo books implementa fallback automático para busca FULLTEXT no MySQL/TiDB quando o OpenSearch está indisponível. O índice derivado é mantido sincronizado pelo OpenSearchIndexCleanupService (reconciliação semanal). Esta decisão garante que /books/search nunca propague 5xx ao cliente, mantendo a funcionalidade de descoberta de livros mesmo durante janelas de degradação de infraestrutura.

- **NR-03:** Cache-null-values=false previne cache poisoning

Cenários afetados: C-01

Descrição técnica: A regra arquitetural cache-null-values=false no Redis garante que resultados vazios ou erros não sejam cacheados, evitando que uma resposta negativa temporária de Neo4j ou MySQL se propague como resultado válido para múltiplos usuários. Combinado com TTLs distintos por tipo de dado (15 min trending, 90 dias parâmetros bayesianos), esta decisão mantém o cache como acelerador de leitura sem risco de poluição de estado.

- **NR-04:** Session affinity + RabbitMQ FanoutExchange garantem chat em ambiente multi-instância

Cenários afetados: C-06

Descrição técnica: O Cloud Run com --session-affinity garante que cada cliente WebSocket permaneça na mesma instância durante toda a conexão STOMP. Para mensagens cross-instance, o WebSocketMessageBroadcastAdapter publica via FanoutExchange e o CommunityBroadcastConsumer entrega em cada instância

ativa. O teste K6 de WebSocket (message domain) validou 100% de entrega sob carga com 0% de falhas 5xx.

Tradeoffs Técnicos (T)

- **T-01:** Latência de Leitura vs. Consistência Eventual (Cache + Bancos Especializados)
Cenários afetados: C-01, C-02
Decisão técnica: Uso de Redis (cache), Neo4j (grafo), OpenSearch (busca) e MySQL (canônico) como fontes de dados distintas para diferentes necessidades de leitura.
Descrição técnica: O uso de cache Redis e bancos especializados reduz drasticamente a latência (p95 < 100ms na maioria dos endpoints; roll-dice com 1.768 req/s) e protege o MySQL de sobrecarga. O custo aceito é a consistência eventual: um dado alterado no MySQL pode levar segundos para refletir no Neo4j, OpenSearch e Redis. Para uma rede social de leitura, essa janela é aceitável — a integridade transacional é preservada no MySQL, e a experiência de leitura é priorizada.
- **T-02:** Segurança de Sessão vs. Disponibilidade de Sessão (Rotação Obrigatória de Refresh Token)
Cenários afetados: C-03, C-05
Decisão técnica: Refresh Token com rotação obrigatória a cada renovação, armazenado em FlutterSecureStorage (mobile) e gerenciado pelo AuthInterceptor.
Descrição técnica: A rotação obrigatória reduz a janela de uso de um token roubado (um RT usado pelo atacante invalida o do usuário legítimo, forçando novo login). O custo é a complexidade no AuthInterceptor: um bug na persistência do novo token desloga o usuário prematuramente — trocando segurança por disponibilidade de sessão. No cenário offline, se o período sem conexão exceder os 7 dias de validade do RT, a sincronização falha silenciosamente até novo login.
- **T-03:** Isolamento de Falhas vs. Consistência Eventual (Comunicação Assíncrona por Eventos)
Cenários afetados: C-04
Decisão técnica: Comunicação entre os 11 módulos exclusivamente via eventos assíncronos (RabbitMQ + Outbox Pattern), nunca por chamada direta entre módulos.
Descrição técnica: O desacoplamento via eventos permite que um módulo com bug não derrube os demais e que novos módulos (ex: T7 de recomendação) sejam adicionados sem alterar código existente — comprovado: o sistema cresceu de 11

para 40 RFs sem reescrita de módulos. O custo é a consistência eventual: eventos BOOK_FINISHED podem levar segundos para disparar a atualização de recomendações, e cada novo consumer deve implementar corretamente a verificação de event_id — um cuidado que hoje recai individualmente sobre cada desenvolvedor.

- **T-04:** Resiliência sem 5xx vs. Lógica de Busca Duplicada (OpenSearch + Fallback FULLTEXT)

Cenários afetados: C-02

Decisão técnica: Manutenção de índice derivado OpenSearch com fallback para FULLTEXT MySQL, reconciliados semanalmente pelo OpenSearchIndexCleanupService.

Descrição técnica: A estratégia elimina erros 5xx de busca originados por degradação de serviços externos, garantindo continuidade funcional. O custo é a duplicação da lógica de busca: duas implementações (OpenSearch e FULLTEXT) precisam ser mantidas e testadas, e resultados via fallback são menos relevantes (sem o scoring de relevância do OpenSearch), impactando a experiência durante a janela de falha.

- **T-05:** Experiência Offline vs. Estado Duplicado (Offline-First Mobile)

Cenários afetados: C-05

Decisão técnica: Repository local-first com Drift/SQLite e Hive; sincronização automática ao reconectar via RetryInterceptor com backoff exponencial.

Descrição técnica: O offline-first elimina a dependência de conectividade constante, endereçando diretamente a realidade de rede instável do público-alvo brasileiro. O custo é a duplicação de estado entre cliente (Drift + Hive) e servidor (MySQL), com lógica de sincronização que não existiria em arquitetura puramente online-only, e a ausência de resolução de conflitos para edições simultâneas offline em múltiplos dispositivos.

Pontos de Sensibilidade (S)

- **S-01:** Latência das 6 trilhas de recomendação e eficácia do cache Redis/Neo4j/OpenSearch

Cenários afetados: C-01

Descrição técnica: O endpoint /recommendations/* dispara 6 estratégias em paralelo, cada uma com suas próprias consultas (Redis, Neo4j 2 níveis, MySQL, OpenSearch). O p95 real sob 500 VUs foi de 773ms — sensível a qualquer degradação nos serviços gerenciados (latência do Upstash Redis, timeout do Neo4j Aura, lentidão do TiDB Cloud). Um aumento de 50ms na latência do Redis

ou Neo4j propaga diretamente ao p95 do endpoint, podendo violar o SLA de 1.200ms definido nos testes K6. A configuração min-co-readers, min-reviews e os TTLs de cache por trilha são parâmetros críticos: reduzir min-co-readers abaixo de 2 aumenta ruído; reduzir TTL aumenta carga nos bancos; aumentar TTL reduz relevância das recomendações.

- **S-02:** Cobertura total do JSoup sobre todos os campos de entrada livre de texto
Cenários afetados: C-03
Descrição técnica: A sanitização XSS via JSoup deve abranger todos os campos de entrada de texto livre da plataforma: posts, comentários, reviews, bio, nome e descrição de comunidade. Qualquer novo campo adicionado por um módulo sem passar pela sanitização reabre a superfície de XSS. Não há um interceptor centralizado que sanitize automaticamente — a responsabilidade recai no desenvolvedor de cada novo endpoint, tornando a cobertura sensível ao processo de code review.
- **S-03:** Constraint de unicidade (userId, trilha) em recommendation_results como contrato compartilhado
Cenários afetados: C-04
Descrição técnica: A tabela recommendation_results e sua constraint de unicidade por (userId, trilha) são o único contrato que todas as 6 trilhas existentes e qualquer nova trilha devem respeitar. Uma migração que altere a estrutura dessa constraint (ex: adicionar partition key, mudar tipo de coluna) impacta simultaneamente as 6 trilhas existentes e todas as futuras — um ponto de acoplamento implícito que não aparece no código dos módulos, mas na DDL compartilhada do schema.sql.
- **S-04:** Validade do Refresh Token (7 dias) como limite do período offline sincronizável
Cenários afetados: C-05
Descrição técnica: A sincronização offline depende de o RT em FlutterSecureStorage ainda ser válido quando a conectividade retornar. Se o período offline exceder 7 dias, o AuthInterceptor não consegue renovar o Access Token via /auth/refresh, e a sincronização falha silenciosamente até um novo login manual — acoplando o atributo de Usabilidade (offline-first) ao mecanismo de Segurança (rotação de RT). Alterar o TTL do RT de 7 para 30 dias aumenta a janela offline sincronizável mas amplia a janela de uso de um RT roubado.

6. Plano de Ação e Recomendações Técnicas

As ações abaixo visam neutralizar os riscos identificados na Seção 5, organizadas por urgência. Cada ação referencia o risco que mitiga e descreve a mudança técnica concreta necessária.

Ações de Curto Prazo (Mitigação Crítica — semanas):

Ação 1 — Mitiga R-01: Implementar lock otimista (@Version) nas entidades Voting e JoinRequest do módulo community. Adicionar campo version Long @Version nas duas entidades; o Spring Data lançará OptimisticLockingFailureException em caso de conflito, que deve ser capturada no serviço e retornada como 409 Conflict ao cliente. Adicionar retry automático no cliente (frontend/mobile) para 409 em votação — o usuário vota sem perceber a contenção. Elimina o risco de escrita silenciosamente incorreta sob alta concorrência.

Ação 2 — Mitiga R-02: Criar suite de testes de contrato de segurança (Spring Security Test) que valide, para cada endpoint REST, que: (a) endpoints protegidos retornam 401 sem token válido; (b) endpoints públicos explícitos retornam 200/400 (nunca 401). Integrar essa suite ao pipeline CI/CD (GitHub Actions) como gate obrigatório antes do deploy — qualquer endpoint marcado incorretamente em SecurityConfig quebra o build antes de chegar a produção.

Ação 3 — Mitiga S-02: Criar um interceptor centralizado (HandlerInterceptor ou AOP @Aspect) que sanitize automaticamente todos os campos String de DTOs de entrada marcados com uma anotação customizada @SanitizeHtml via JSoup, eliminando a dependência de cada desenvolvedor lembrar de aplicar manualmente a sanitização em novos campos.

Ações de Médio/Longo Prazo (Sustentabilidade — meses):

Ação 4 — Mitiga R-03: Migrar o OpenSearch de produção da VM GCE própria (e2-small, operação manual) para um serviço gerenciado com auto-healing — opções: Bonsai.io Standard (~\$20/mês), OpenSearch Service na AWS ou Elastic Cloud. Elimina o ponto de operação não gerenciado, adiciona backups automáticos e remove a dependência de SSH manual para recuperação. Custo estimado: \$20–\$40/mês coberto pelo crédito GCP ou orçamento de produção.

Ação 5 — Mitiga R-04: Implementar mecanismo de last-write-wins com timestamp de cliente (clientUpdatedAt) em ShelfItem. O backend compara o timestamp do payload com o updatedAt persistido; se o payload for mais antigo, retorna 409 com o estado atual — o app mobile exibe o conflito ao usuário e permite escolha. Curto prazo: adicionar updatedAt ao DTO e validação no serviço. Longo prazo: avaliar CRDT para campos numéricos (progresso de página).

Ação 6 — Mitiga S-01: Implementar Circuit Breaker (Resilience4j) nas chamadas de

recomendação ao Neo4j Aura e ao Upstash Redis. Se o neo4j-java-driver detectar timeout consecutivo, o circuit breaker abre e o fallback para SQL é ativado automaticamente, limitando a propagação de latência dos serviços gerenciados externos para o p95 do endpoint de recomendações.

Ação 7 — Evolução Arquitetural: Implementar testes de caos (Chaos Engineering com Gremlin ou scripts de injeção de falha) para validar empiricamente os fallbacks de OpenSearch → FULLTEXT e Neo4j → SQL em ambiente de produção. Os testes K6 existentes cobrem carga, mas não cobrura de falha parcial de infraestrutura — os cenários C-02 e C-06 ainda dependem de validação manual.

Signed by:

Marcos Alberto Ferreira Pinto

2C40C02291E94D8...

MARCOS ALBERTO
ARQUITETO AVALIADOR

Signed by:

Marcos Alberto Ferreira Pinto

2C40C02291E94D8...

MARCOS ALBERTO
ARQUITETO DO SISTEMA

Signed by:

Bernardo S. Alvim

AB03A904D888429...

Signed by:

Gabriela Alvaranga

169AFB31B211458...

Assinado por:

Rafael Ganascini de

5751AD815E7B4F3...

Signed by:

Carlos José Figueiredo

9BA57DE373BF413...

Signed by:

Mateus Santos

89D1FAE633F84E9...

EQUIPE BIBLIOO
PRODUCT OWNER OU CTO